GameStats

Arquitectura, decisiones de diseño y solución al problema

Plataforma multilenguaje que recibe los eventos de partidas de videojuegos y los convierte en estadísticas. **Go** se encarga de la ingestión imperativa y concurrente, y **Scala** del análisis funcional.

Curso: Lenguajes de Programación

Repositorio: github.com/kayy1y/ProyectoLenguajes

Fecha del documento: 14 de octubre de 2026

Contenido

1. El problema y lo que pide el enunciado
2. La solución en una página
3. Arquitectura general
4. Juegos elegidos y diseño de los eventos
5. Go: ingestión imperativa y concurrente
6. Scala: análisis funcional
7. Estadísticas y patrones, con un ejemplo calculado a mano
8. Extensibilidad: cómo se agrega un juego
9. Registro de decisiones de arquitectura
10. Imperativo frente a funcional: ventajas y limitaciones
11. Calidad, contenedores, CI/CD y despliegue
12. Herramientas para probar el sistema
13. Limitaciones conocidas
14. Preguntas probables en la defensa

> **Cómo leer este documento.** Las secciones 1–3 dan la visión completa en pocas páginas. Las secciones 4–8 explican el diseño de cada parte y por qué funciona así. Las secciones 9–10 justifican las decisiones; son la base de la defensa.

1. El problema y lo que pide el enunciado

Un videojuego produce continuamente hechos: jugadores que entran, ataques, vueltas completadas, desconexiones, finales de partida. El reto es construir una plataforma que reciba ese flujo y produzca información útil, no solo conteos: estadísticas por jugador y por partida, métricas que relacionen tipos de evento distintos, y la detección de situaciones que dependen de una *secuencia* de eventos.

El enunciado fija una distribución de responsabilidades obligatoria entre dos lenguajes y dos paradigmas:

Go — imperativo

- Recibir los eventos de una fuente externa.
- Validar su estructura y tratar los inválidos.
- Mantener el estado de cada partida y agrupar sus eventos.
- Procesar varias partidas y juegos en paralelo con goroutines y channels.
- Preparar los datos y enviarlos a Scala.

Scala — funcional

- Recibir los datos preparados por Go.
- Transformarlos con colecciones y operaciones funcionales.
- Calcular las estadísticas definidas por el equipo.
- Detectar patrones basados en secuencias de eventos.
- Producir resultados estructurados (JSON).

Las diez funcionalidades mínimas y cómo se cumplen

#	Funcionalidad pedida	Cómo la resuelve GameStats
1	Registrar o recibir eventos	`POST /v1/events` en Go; el simulador actúa como fuente externa
2	Validar y procesar eventos	JSON Schema por juego; los inválidos responden `400` con campo y código, y quedan registrados en `/v1/rejections`
3	Identificar juegos, partidas y jugadores	La identidad de una partida es `(gameId, matchId)`; `playerId` es obligatorio en los eventos de jugador
4	Procesar varios eventos de una misma partida	Una goroutine por partida acumula, ordena y deduplica sus eventos
5	Calcular estadísticas propias	Analizadores en Scala para dos juegos
6	Estadística por jugador	Precisión, daño, eliminaciones, mejor vuelta, entre otras
7	Estadística por partida	Duración, totales, mejor jugador, vuelta más rápida, ganador
8	Estadística que combine tipos de evento	`killEfficiency` (eliminaciones ÷ ataques), eliminaciones por minuto, `positionsGained`
9	Regla basada en una secuencia de eventos	`KILL_STREAK` (racha sin morir) y `COMEBACK` (remontada)
10	Consultar o presentar los resultados	`GET …/results` en JSON, una aplicación de escritorio y la terminal

Además se cubren los requisitos de desarrollo y entrega: repositorio Git con historial incremental, construcción reproducible, un contenedor Docker por módulo, Docker Compose, CI/CD en GitHub Actions

y un procedimiento de despliegue documentado.

2. La solución en una página

GameStats separa **recibir y organizar** de **analizar**. Go recibe eventos sueltos, desordenados y a veces erróneos, y los convierte en partidas completas, ordenadas y limpias. Scala recibe cada partida completa como un valor inmutable y la transforma en estadísticas mediante una función pura.

El recorrido de un evento

1. **Llega** un evento JSON a `POST /v1/events`. El productor puede ser el simulador o cualquier sistema externo que siga la especificación.
2. **Go lo valida** contra el esquema de su juego. Si es inválido, responde `400` indicando el campo y el motivo, y lo anota en el registro de rechazos.
3. **El dispatcher** lo entrega a la goroutine de su partida, y la crea si es el primer evento de esa partida.
4. **El worker de la partida** descarta los duplicados, le asigna un número de llegada y actualiza el estado de la partida.
5. Cuando llega `MATCH_FINISHED`, el worker **espera 2 segundos** (período de gracia) por si llegan eventos rezagados. Si una partida deja de recibir eventos, se cierra sola por inactividad.
6. **El pool de análisis** envía el lote ordenado a Scala (`POST /v1/analyze`), con reintentos si Scala no responde.
7. **Scala** ordena, tipa y filtra los eventos, y calcula las métricas y los patrones. Devuelve un JSON determinista.
8. **Go guarda** el resultado sin interpretarlo y lo expone en `GET /v1/games/{juego}/matches/{partida}/results`.

Ideas clave del diseño

Idea	En qué consiste
Contrato primero	Los esquemas JSON, los ejemplos y los resultados esperados en `contracts/` son la fuente de verdad. Go valida con ellos y las pruebas de los dos lenguajes los usan.
Estado confinado	Cada partida tiene una sola goroutine dueña de su estado. No hay locks sobre los eventos y el orden está garantizado.
Análisis puro	La misma partida produce siempre el mismo resultado, sin importar el orden de llegada de sus eventos. Esto permite reintentar sin riesgo.
Nada inventado	Lo que no se puede calcular vale `null`, nunca un 0 ni `NaN`. Los eventos incoherentes se excluyen con un aviso, en lugar de hacer fallar el análisis.
Genérico por diseño	Agregar un juego es agregar un esquema, un analizador y una línea en el registro. Ni Go ni el núcleo de Scala cambian.

> **Resultado verificable.** Ocho partidas de prueba tienen su resultado calculado a mano. El sistema completo (Go → Scala) debe reproducirlo exactamente, y lo hace: se comprueba en las pruebas de Scala, en la prueba de punta a punta sobre Docker y en la suite de verificación de 37 comprobaciones.

3. Arquitectura general

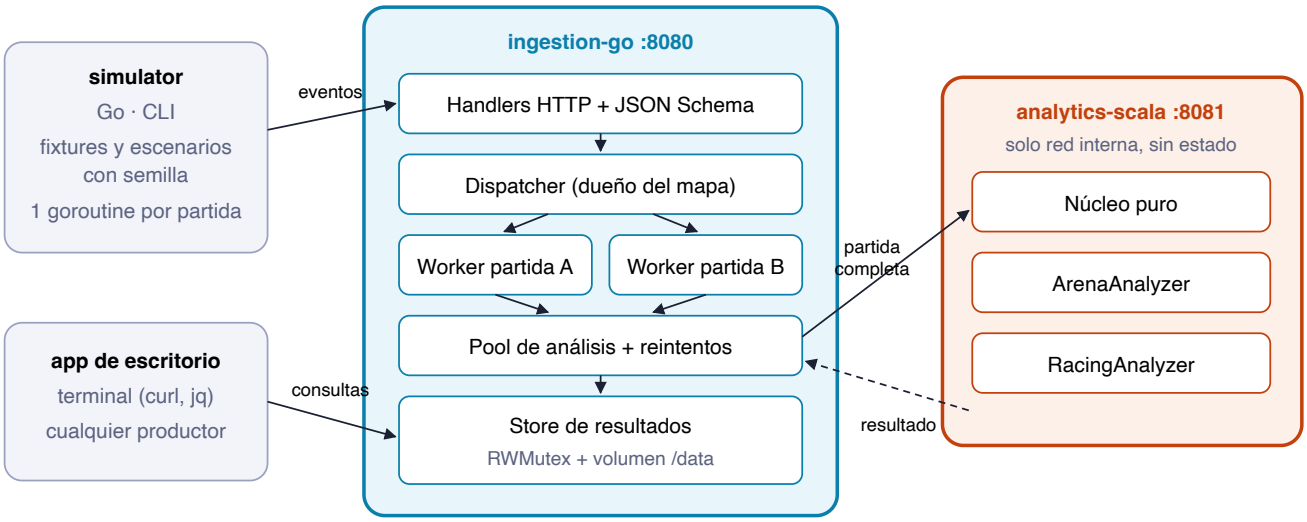


Figura 1. Componentes y flujo de datos. Solo el puerto 8080 queda expuesto; Scala vive en la red interna de Docker.

Componente	Paradigma	Responsabilidad	Lo que no hace
ingestion-go	Imperativo y concurrente	Recibe, valida, deduplica y agrupa por partida; controla el ciclo de vida; coordina el análisis; guarda y expone los resultados	Calcular métricas o patrones de juego
analytics-scala	Funcional	Ordena, tipa, aplica reglas semánticas, calcula métricas y detecta patrones	Guardar estado, coordinar concurrencia, recibir eventos sueltos
simulator	Go	Simula la fuente externa: varias partidas de dos juegos en paralelo, con casos especiales	Calcular estadísticas
contracts/	—	Esquemas, ejemplos válidos e inválidos, partidas de prueba y resultados calculados a mano	—

¿Por qué dos servicios que se comunican por HTTP?

El enunciado exige que los módulos se ejecuten como **componentes independientes**. Cada uno corre en su propio contenedor y se comunica mediante un protocolo documentado:

- **Simulador → Go:** un evento por petición.
- **Go → Scala:** una partida completa por petición.

HTTP/REST se eligió por dos razones. Primero, el contrato es explícito y se prueba con una sola línea de `curl`. Segundo, los healthchecks permiten que Docker Compose arranque los servicios en orden: Scala primero y Go cuando Scala está sano. Una cola de mensajes habría agregado un tercer servicio sin un beneficio que se pueda evaluar en este proyecto.

Go no interpreta el resultado de Scala: lo guarda como bytes opacos (`json.RawMessage`). Así queda garantizado que **todas** las estadísticas de juego vienen de Scala. Los únicos conteos que hace Go son de diagnóstico de la ingestión (aceptados, rechazados, duplicados), no del juego.

4. Juegos elegidos y diseño de los eventos

El enunciado deja libre la elección de juegos, pero pide que el núcleo no dependa de uno solo. Por eso se eligieron dos juegos de naturaleza **muy distinta**. Un juego de combate y uno de carreras no comparten casi ningún evento propio, así que si ambos funcionan sobre el mismo núcleo, la generalidad queda demostrada en la práctica y no solo prometida.

> **arena-combat**: combate todos contra todos, con reaparición.
>
> Eventos propios: `ATTACK` (objetivo, arma, acierto, daño), `ELIMINATION` (víctima, arma) y `OBJECTIVE_CAPTURED`.
>
> Permite medir eficacia (precisión, daño), dominio (eliminaciones, muertes) y rachas, que son un patrón secuencial natural.

> **street-racing**: carrera por vueltas.
>
> Eventos propios: `LAP_COMPLETED` (vuelta, tiempo, válida), `POSITION_CHANGED` y `PLAYER_FINISHED`. `PLAYER_JOINED` lleva la posición de salida.
>
> Permite medir ritmo (mejor vuelta, promedio), resultado y remontadas, otro patrón que depende del orden.

Los dos comparten cuatro eventos de **ciclo de vida** que entiende el núcleo: `MATCH_STARTED`, `PLAYER_JOINED`, `PLAYER_LEFT` y `MATCH_FINISHED`.

Por qué estos eventos y no otros

- **Los eventos describen hechos, nunca estadísticas.** Ningún evento trae `kills`, `accuracy` ni `bestLap`: todo eso lo deriva Scala. Si un productor envía un campo así, el esquema lo rechaza (`UNKNOWN_FIELD`).
- **Cada evento lleva lo mínimo para relacionarse con otros.** Por ejemplo, `ATTACK` y `ELIMINATION` nombran al jugador afectado. Eso permite la eficiencia (relación entre tipos) y la racha (relación en el tiempo).
- **En carreras no basta la posición final.** Se registran las vueltas (con su validez) y los cambios de posición. Así se puede calcular el ritmo y detectar una remontada, como sugiere el propio enunciado.

Formato común de un evento

```
{
  "eventId": "a1-e05",                identificador único dentro de la partida
  "timestamp": "2026-10-01T15:00:10Z", instante del hecho (UTC, ISO 8601)
  "gameId": "arena-combat",           juego …
  "gameVersion": "1.0",               … y versión de su catálogo
  "matchId": "match-a1",              partida (identidad = gameId + matchId)
  "playerId": "p1",                   solo en eventos de jugador
  "type": "ATTACK",                   tipo, dentro del catálogo del juego
  "seq": 5,                           opcional: desempata eventos con el mismo timestamp
  "data": { "targetPlayerId": "p2", "weapon": "rifle", "hit": true, "damage": 40 }
}
```

Respecto a la propuesta del enunciado, se fusionaron `type` y `action` en un solo campo `type`. Así se evitan combinaciones inválidas de tipo y acción, y la validación queda más simple.

Validación: JSON Schema como contrato

Cada juego tiene un esquema (`contracts/schemas/games/*.schema.json`) que extiende el sobre común y define, para cada tipo, si `playerId` es obligatorio o está prohibido, y la forma exacta de `data`. Go valida cada evento con su esquema y traduce los errores a códigos estables. La prueba exige que el **primer** error reportado sea el motivo real:

Código	Ejemplo
`REQUIRED_FIELD`	Falta `eventId`, o un `ATTACK` sin `playerId`
`INVALID_FORMAT`	Timestamp mal escrito, imposible o sin zona UTC
`INVALID_TYPE` / `OUT_OF_RANGE`	`damage: "veinte"` o `damage: -5`
`UNKNOWN_GAME` / `UNSUPPORTED_GAME_VERSION` / `UNKNOWN_EVENT_TYPE`	Juego, versión o tipo que no existen
`FORBIDDEN_FIELD` / `UNKNOWN_FIELD`	`MATCH_FINISHED` con `playerId`, o un campo extra en `data`
`MALFORMED_JSON`	El cuerpo no es JSON

Hay 17 ejemplos inválidos, cada uno con el código y el campo que debe producir, y se usan como pruebas. Con esa especificación, un productor externo puede generar eventos sin conocer la implementación, como pide el enunciado.

5. Go: ingestión imperativa y concurrente

La ingestión es un problema de **tiempo, coordinación y estado**: llegan eventos de muchas partidas a la vez, desordenados, repetidos o tarde, y hay que decidir cuándo cada partida está completa. Por eso el control explícito del flujo que da el estilo imperativo encaja aquí.

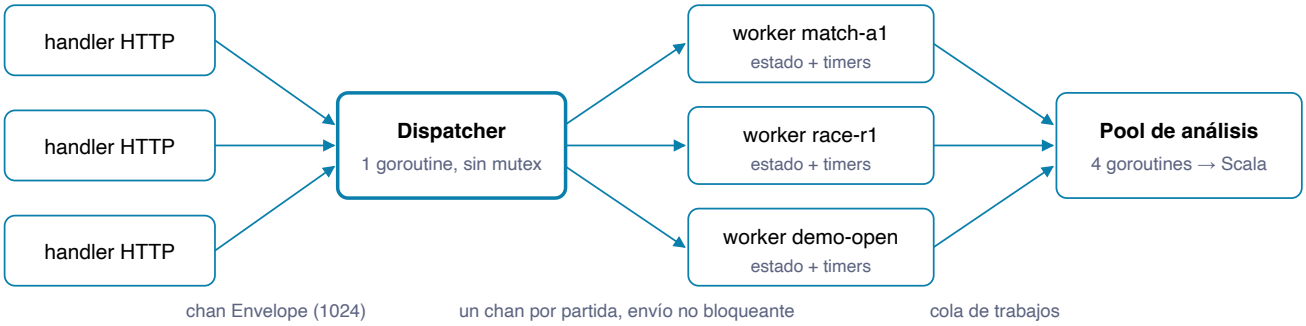


Figura 2. Topología de goroutines y canales en ingestion-go.

Goroutine	Cantidad	Es dueña de	Por qué así
Handler HTTP	1 por petición	nada	Valida (no tiene estado) y delega; espera la respuesta en un canal propio
Dispatcher	**1**	el mapa de partidas y las partidas cerradas	Al ser el único que toca el mapa, no necesita mutex (**confinamiento**)
Worker de partida	**1 por partida**	eventos, duplicados, estado y temporizadores	Serializa los eventos de su partida sin locks; partidas distintas avanzan en paralelo
Pool de análisis	4	el trabajo en curso	Las llamadas a Scala son lentas (I/O) y no deben frenar la ingestión
Store	—	los resultados	Única estructura compartida, protegida con `sync.RWMutex`

Mecanismos y su justificación

- **Orden total.** Cada evento se ordena por `(timestamp, seq, orden de llegada)`. Es una clave determinista que no inventa causalidad cuando dos eventos tienen el mismo instante.
- **Duplicados idempotentes.** Go guarda un hash SHA-256 del contenido de cada `eventId`:
  - mismo id y mismo contenido: `200 DUPLICATE`, y el evento no se cuenta dos veces;
  - mismo id con otro contenido: `409 DUPLICATE_ID_CONFLICT`.

  Así el productor puede reintentar con seguridad.
- **Período de gracia.** Tras `MATCH_FINISHED` se esperan 2 s para absorber eventos rezagados por la red. Lo que llega después recibe `409 LATE_EVENT` y queda visible en los rechazos.
- **Cierre por inactividad.** Una partida sin eventos durante el tiempo configurado se analiza como `INCOMPLETE`. Scala no inventa entonces la duración: vale `null`.
- **Backpressure sin bloqueo global.** El dispatcher entrega con envío no bloqueante. Si la cola de una partida está llena, solo esa partida recibe `503` y las demás siguen.

- **Reintentos.** Si Scala falla (5xx, timeout, conexión rechazada), Go reintenta hasta 3 veces con backoff exponencial y jitter. Un error 4xx es un error de contrato y no se reintenta. Si se agotan los intentos, la partida pasa a `FAILED` y conserva su lote para `POST …/reanalyze`.
- **Persistencia.** Los resultados se guardan con escritura atómica en un volumen (`/data`) y sobreviven a un reinicio. Una partida recargada no se reabre.
- **Apagado ordenado.** Primero se deja de aceptar peticiones. Luego cada worker envía su partida (las abiertas, como `INCOMPLETE/SHUTDOWN`). Por último, el pool termina sus trabajos antes de salir.

Ciclo de vida de una partida

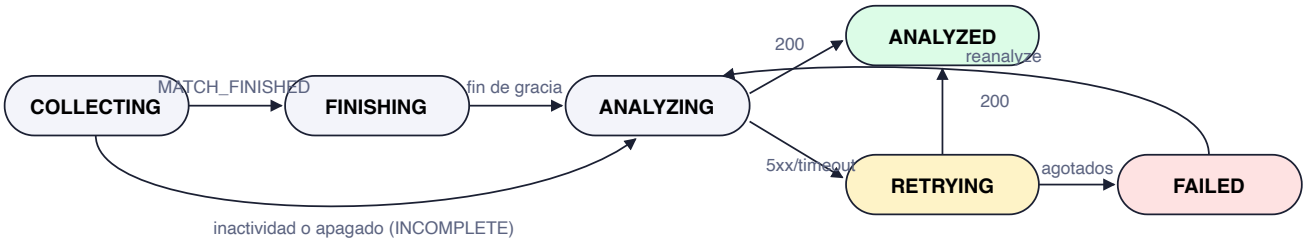


Figura 3. Estados de una partida. Después de ANALYZING, cualquier evento nuevo se rechaza (LATE_EVENT o MATCH_CLOSED).

6. Scala: análisis funcional

Analizar una partida es, por naturaleza, una **función**: entra la lista de eventos y sale un resultado. No hay estado que recordar entre peticiones ni concurrencia que coordinar. Por eso el estilo funcional encaja: datos inmutables, transformaciones de colecciones y composición de funciones.

El pipeline

```
RawEvent ──ordered──▶ ──decode(juego)──▶ Either[Aviso, Event[P]] ──semantics──▶ (conservados, avisos)
         ──context──▶ MatchContext[P] ──playerMetrics / matchMetrics / patterns──▶ AnalysisResult
```

- `core/Analysis.scala` compone las etapas con `andThen`. Los errores de la petición (lote vacío, juego no soportado) viajan en un `Either` dentro de un for-comprehension, sin excepciones.
- **Decodificación tipada:** cada juego convierte el JSON genérico en su propio tipo algebraico, por ejemplo `Attack`, `Elimination` o `LapCompleted`. A partir de ahí el compilador garantiza que el análisis cubre todos los casos.
- **Reglas semánticas:** son las comprobaciones que necesitan la partida completa, como una eliminación de alguien que nunca se unió o una autoeliminación. Generan un aviso y excluyen el evento, en lugar de hacer fallar el análisis.
- `core/GameAnalyzer.scala` es la única interfaz que implementa un juego: `decode`, `check`, `playerMetrics`, `matchMetrics` y `patterns`. El núcleo calcula una sola vez lo común: duración, participantes y total de eventos.

Técnicas funcionales usadas y dónde

Técnica	Uso concreto
`foldLeft`	Contadores por jugador en una sola pasada (`Tally` inmutable); estado de las rachas sin variables (`KillStreak`); reglas semánticas
`collect`	Filtrar y proyectar a la vez sobre un tipo algebraico (por ejemplo, quedarse solo con las vueltas)
`groupMapReduce`, `groupMap`	Conteos por clave y vueltas agrupadas por piloto
`partition`, `partitionMap`	Vueltas válidas frente a inválidas; eventos decodificados frente a avisos
`sortBy` con tupla	El orden total `(timestamp, seq, llegada)`
`Option`, `minOption`	Métricas no calculables: `safeRatio(n, 0) = None`, que se serializa como `null`
`andThen`	Composición de las etapas del pipeline
Inmutabilidad	`case class` y `copy`; no hay `var` ni colecciones mutables en el análisis

> **Transparencia referencial, demostrada con una prueba.** Una prueba de propiedades desordena al azar los eventos de cada partida de prueba, 50 veces por partida, y verifica que el resultado sea exactamente el mismo. También comprueba invariantes; por ejemplo, la suma de eliminaciones por jugador debe ser igual al total de la partida. Esto es posible gracias a que el análisis es una función pura, y es lo que permite que Go reintente sin riesgo.

Bibliotecas. Cask provee la capa HTTP, muy delgada; upickle hace la serialización JSON y munit las pruebas. Se descartó un stack de efectos (http4s y cats-effect): habría desplazado el foco académico hacia la programación de efectos, cuando lo que pide el enunciado son las transformaciones de colecciones.

7. Estadísticas y patrones, con un ejemplo calculado a mano

	arena-combat	street-racing
Por jugador	ataques, aciertos, precisión, daño, eliminaciones, muertes, objetivos	vueltas válidas e inválidas, mejor vuelta, promedio, posición de salida y de llegada
Por partida	duración, participantes, eliminaciones y daño totales, mejor jugador	finalistas, abandonos, vuelta más rápida y su piloto, ganador
Combinada	`killEfficiency` = eliminaciones ÷ ataques; eliminaciones por minuto = eliminaciones ÷ (duración/60)	`positionsGained` = salida − llegada
Patrón secuencial	`KILL_STREAK` : 3 o más eliminaciones seguidas sin morir ni salir	`COMEBACK` : estar 3º o peor y después ganar

Por qué los patrones no son conteos

- **KILL_STREAK** se calcula con un `foldLeft` sobre los eventos ordenados, con un estado `Map[jugador, racha abierta]` :
  - una eliminación extiende la racha del que elimina y *cierra* la de la víctima;
  - salir de la partida cierra la racha;
  - se emiten las rachas maximales: una racha de 4 es un patrón de longitud 4, no dos de 3.

  En la partida F12 el jugador p1 tiene 3 eliminaciones, igual que en F01, pero muere entre la segunda y la tercera: **no hay patrón**.
- **COMEBACK** recorre la trayectoria de posiciones de cada piloto: la posición de salida y luego cada cambio, en orden. Exige haber estado 3º o peor **antes** de llegar primero. Ganar saliendo desde la primera posición no cuenta (F18).

Ejemplo completo: partida F01 (`match-a1`)

Tres jugadores; la partida dura de T+0 a T+120 s. Eventos relevantes, en orden:

Evento	t	Hecho	Evento	t	Hecho
e05	10	p1 ataca a p2: acierto, 40	e12	40	p1 ataca a p2: acierto, 100
e06	12	p1 ataca a p2: acierto, 60	e13	40	**p1 elimina a p2 → racha de 3**
e07	12	**p1 elimina a p2**	e14	50	p2 captura flag-A
e08	20	p2 ataca a p1: fallo	e15	55	p2 ataca a p1: acierto, 100
e09	25	p1 ataca a p3: acierto, 100	e16	55	p2 elimina a p1 → se cierra la racha
e10	25	**p1 elimina a p3**	e17	60	p1 ataca a p3: fallo
e11	30	p3 ataca a p1: acierto, 30	e18	120	MATCH_FINISHED

Jugador	Ataques	Aciertos	Precisión	Daño	Elim.	Muertes	Eficiencia
p1	5	4	4/5 = 0.8	40+60+100+100 = 300	3	1	3/5 = 0.6
p2	2	1	0.5	100	1	2	0.5

p3	1	1	1.0	30	0	1	0/1 = 0

Partida: 120 s, 3 participantes, 18 eventos, 4 eliminaciones, 430 de daño, 4 ÷ 2 min = 2 eliminaciones por minuto; el mejor jugador es p1. **Patrón:** `KILL_STREAK` de p1, de e07 a e13, longitud 3.

F06 envía exactamente estos eventos **desordenados**, uno de ellos incluso después de `MATCH_FINISHED` pero dentro de la gracia. El resultado es idéntico: el orden lo da el timestamp, no la llegada.

Las ocho partidas de prueba

Fixture	Qué demuestra
F01	Caso normal (tablas de arriba)
F06	Llegada desordenada: el resultado es idéntico a F01
F10	Denominador cero: precisión y eficiencia en `null` para quien no atacó
F11	Racha maximal de 4 y una segunda racha de 3
F12	Racha interrumpida por una muerte: sin patrón
F13	Eventos contradictorios: 4 avisos y 4 de 10 eventos excluidos
F17	Remontada de d3 (sale 3º y gana), con vueltas inválidas que no cuentan
F18	Gana desde la primera posición (sin remontada) y un abandono

8. Extensibilidad: cómo se agrega un juego

El enunciado exige que un nuevo conjunto de eventos se pueda incorporar sin modificar completamente el sistema. En GameStats, agregar un juego requiere cuatro piezas:

1. **Un esquema** en `contracts/schemas/games/<juego>.v1.schema.json`. Go lo carga solo al arrancar.
2. **Un analizador** en Scala que implemente `GameAnalyzer[P]`, con su propio tipo de eventos.
3. **Una línea** en `core/Registry.scala`.
4. **Una partida de prueba** con su resultado esperado calculado a mano.

> **Evidencia en el historial de Git.** El juego `street-racing` se agregó después de `arena-combat`, en el commit `feat(racing)`. Ese commit no modifica ningún archivo de la ingestión en Go ni del núcleo de Scala, salvo la línea del registro. Se comprueba con `git show --stat`. Antes hubo un único cambio en el núcleo, en un commit aparte: conservar el `data` de `MATCH_STARTED` y `PLAYER_JOINED`, para que cada juego pueda leer atributos propios como la posición de salida.

Las herramientas de prueba también son genéricas:

- La aplicación de escritorio descubre los juegos leyendo los esquemas.
- Sus tablas toman las columnas de las claves de `metrics` del resultado.

Un tercer juego aparecería en ellas sin tocar su código.

Separación de responsabilidades por capa

Si cambia…	Se toca	No se toca
Un juego nuevo	Esquema, analizador, registro, fixture	Go completo, núcleo de Scala, simulador, app
Una métrica nueva de un juego	Su analizador y su resultado esperado	Go, el resto de juegos
Un campo nuevo en un evento	El esquema del juego (nueva versión) y su analizador	Go: valida con el esquema nuevo sin cambiar código
El transporte (por ejemplo, una cola en lugar de HTTP)	El cliente de análisis de Go y la capa HTTP de Scala	Validación, workers y todo el análisis

9. Registro de decisiones de arquitectura

Todas son decisiones del equipo, no impuestas por el enunciado. Cada una indica qué se descartó y qué consecuencia tiene.

ADR-01 · REST síncrono entre Go y Scala

Decisión	Go envía el lote completo de una partida cerrada a `POST /v1/analyze` y espera la respuesta.
Motivo	Contrato explícito, fácil de probar, sin infraestructura extra.
Descartado	Una cola de mensajes (otro servicio sin beneficio evaluable) y archivos compartidos (sondeo y mala retroalimentación de errores).
Consecuencia	Go implementa timeouts, reintentos con backoff, el estado `FAILED` y el reanálisis manual.

ADR-02 · Núcleo genérico, un juego completo y un segundo incremental

Decisión	`arena-combat` completo primero; `street-racing` después, como prueba de extensibilidad.
Motivo	Las abstracciones nacen del primer juego real y se validan con el segundo.
Descartado	Un solo juego (extensibilidad solo prometida) o dos desde el inicio (abstracción prematura).
Consecuencia	La interfaz `GameAnalyzer` y el esquema por juego quedaron probados en la práctica.

ADR-03 · Dispatcher único, un worker por partida y un pool de análisis

Decisión	Una goroutine es la única dueña del mapa de partidas; cada partida tiene su goroutine; las llamadas a Scala van en un pool aparte.
Motivo	Orden garantizado por partida sin locks, paralelismo entre partidas, y una partida lenta no bloquea a las demás.
Descartado	Un mapa con mutex global (contención y riesgo de carreras) o un pool asignado por hash (una partida lenta frena a las que comparten worker).
Consecuencia	Más goroutines, que en Go son baratas, y un diseño fácil de explicar y de probar con `-race`.

ADR-04 · JSON Schema como contrato único

Decisión	`contracts/schemas` es la fuente de verdad: Go valida con ella y las pruebas y la documentación la usan.
Motivo	Un productor externo puede generar eventos solo con la especificación, y agregar un juego es agregar un esquema.
Descartado	Validación escrita a mano en Go, que habría que cambiar con cada juego.
Consecuencia	Dependencia de una biblioteca de JSON Schema en Go.

ADR-05 · Scala sin estado; Go guarda los resultados

Decisión	Scala solo calcula. Go conserva el estado y los resultados en memoria, con copia en disco en un volumen.
Motivo	Separa las responsabilidades: el estado y la coordinación son imperativos, el cálculo es puro. No hace falta una base de datos.
Descartado	Una base de datos persistente.
Consecuencia	Al reiniciar se pierden las partidas en curso y los lotes fallidos, y así está documentado. Los resultados analizados sí persisten.

ADR-06 · Análisis al cierre, con período de gracia

Decisión	Tras `MATCH_FINISHED` se esperan 2 s antes de analizar. Lo que llega después recibe `409 LATE_EVENT`.
Motivo	Es el comportamiento más simple que sigue siendo coherente frente a eventos rezagados.
Descartado	Reanálisis incremental en cada evento tardío.
Consecuencia	Los eventos muy tardíos se pierden, pero quedan visibles en `/v1/rejections`.

ADR-07 · Orden total (timestamp, seq, llegada)

Decisión	Se ordena por instante, luego por la secuencia del productor (si falta, va al final) y luego por orden de llegada.
Motivo	Un orden determinista que no inventa causalidad entre eventos simultáneos.
Descartado	Ordenar solo por timestamp, que no es determinista ante empates.
Consecuencia	Se recomienda a los productores enviar `seq`.

ADR-08 · Scala 3 con Cask, upickle y munit

Decisión	Capa HTTP delgada; todo el análisis es puro.
Motivo	Curva de aprendizaje baja; el foco queda en colecciones y funciones puras, que es lo que evalúa el curso.
Descartado	http4s, cats-effect y circe.
Consecuencia	Menos programación de efectos y más transformaciones de colecciones.

ADR-09 · Simulador en Go, como módulo separado

Decisión	`simulator/` tiene su propio módulo, fuera de la ingestión.
Motivo	Un lenguaje menos que mantener, y goroutines para enviar partidas en paralelo.
Descartado	Un simulador en Python.
Consecuencia	El simulador es una fuente externa real: solo conoce la API pública.

ADR-10 · Despliegue en una VM con Compose e imágenes de GHCR

Decisión	CI publica las imágenes en GitHub Container Registry, y el servidor las descarga con Docker Compose.
Motivo	Barato e idéntico al entorno local.
Descartado	Plataformas PaaS: la red privada entre servicios suele ser de pago y los arranques en frío rompen la demo.
Consecuencia	El despliegue es manual y documentado, o con un clic desde `deploy.yml`.

ADR-11 · Aplicación de escritorio como herramienta de apoyo

Decisión	Una ventana nativa (Fyne, en Go) para ver resultados, generar datos y verificar. Se compila aparte, con `-tags gui`.
Motivo	Facilita la evaluación sin terminal y sin cambiar la arquitectura: solo usa la API pública.
Descartado	Una consola web (agregaba un cuarto contenedor y un proxy) y lógica de juego en la interfaz.
Consecuencia	Requiere un compilador de C para compilarla; CI la publica ya compilada para Linux, macOS y Windows.

10. Imperativo frente a funcional: ventajas y limitaciones

El objetivo académico del proyecto es comparar los dos paradigmas sobre un mismo problema. En GameStats cada uno resolvió la parte que mejor le encaja, y los dos tienen costos.

Aspecto	Go — imperativo	Scala — funcional
Estado	Mutable pero **confinado**: cada partida tiene una sola goroutine dueña	Inmutable: `case class` y `copy`, sin `var`
Flujo	Bucle `for { select { … } }` con canales y temporizadores	Pipeline de funciones compuestas con `andThen`
Errores	Códigos HTTP, reintentos, estados `FAILED`	`Either` para errores de la petición; `Option` en lugar de `null` o `NaN`
Ventaja principal	Control exacto de quién toca qué y cuándo; ideal para tiempo, I/O y concurrencia	Misma entrada, mismo resultado: fácil de probar, de razonar y de reintentar
Limitación	Hay que razonar sobre carreras y bloqueos (se mitigan con confinamiento, envío no bloqueante y `-race`); hay más código de coordinación	Menos natural para tiempo y efectos; cada pasada crea colecciones nuevas, un costo aceptable para partidas de cientos o miles de eventos

Por qué la ingestión es imperativa. Recibir eventos implica decidir en el momento, con temporizadores, si una partida terminó, si un evento es duplicado o si una cola está llena. Son decisiones con estado que cambia en el tiempo. Mutar ese estado de forma controlada, dentro de una sola goroutine por partida, es más simple y más eficiente que modelarlo de forma pura.

Por qué el análisis es funcional. Una vez que la partida está completa, las estadísticas son transformaciones de una colección: filtrar, agrupar, reducir. Expresarlas como funciones puras las hace verificables: el mismo cálculo da el mismo resultado sin importar el orden de llegada, lo que se demostró con 50 permutaciones por partida.

Conclusión. La frontera entre los dos módulos coincide con la frontera entre *coordinar en el tiempo* y *calcular sobre datos completos*. Por eso la división de responsabilidades del enunciado no es arbitraria: cada paradigma resolvió con naturalidad su parte, y el contrato JSON entre ellos mantiene a los dos independientes.

11. Calidad, contenedores, CI/CD y despliegue

Pruebas

Nivel	Cantidad	Qué cubre
Go, ingestión (unitarias y de sistema, con `-race`)	19	Validación de los 17 inválidos, duplicados, gracia, inactividad, 4 partidas con 4 productores concurrentes, análisis lento aislado, reintentos, fallos de contrato, reinicio y apagado ordenado
Go, simulador y lógica de la app	14	Semilla determinista, catálogo, comparación con lo calculado a mano, trabajos
Aplicación de escritorio	2	La ventana se construye; capturas y suite completa contra el sistema real
Scala	42	Resultados idénticos a los esperados, 50 permutaciones por partida, rachas, casos límite, HTTP
De punta a punta sobre Docker	23	Fixtures, inválidos, duplicados, tardíos, caída de Scala con recuperación, reinicio de Go, 30 partidas en paralelo
Suite de verificación (app)	37	8 fixtures, 17 inválidos, 12 comportamientos

Contenedores

- Un Dockerfile multietapa por módulo, construido con compilación cruzada para amd64 y arm64.
- Imágenes finales mínimas: distroless sin shell para Go (unos 15 MB) y un JRE alpine para Scala. Los dos corren con un usuario no root.
- Docker Compose define los healthchecks y el orden de arranque (Go espera a que Scala esté sano). También monta un volumen para los resultados y deja el simulador en un perfil aparte.

CI/CD (GitHub Actions)

Workflow	Cuándo	Qué hace
`ci.yml`	Cada push y PR	Formato, `vet`, compilación, pruebas con `-race`, pruebas de Scala, construcción de las 3 imágenes, prueba de punta a punta y ejecutable de escritorio para 3 sistemas
`release.yml`	Push a `main` y tags	Publica las imágenes multiarquitectura en GHCR (entrega continua de artefactos)
`deploy.yml`	Manual	Por SSH, actualiza la versión en el servidor y comprueba `/healthz` desde fuera

Despliegue

En una VM Linux con Docker se copian `deploy/docker-compose.deploy.yml` y un `.env` con la clave de API, y se ejecuta `docker compose up -d`. El servidor usa las imágenes publicadas, así que no compila nada.

- **Exposición:** solo queda abierto el puerto 8080.
- **Seguridad:** las escrituras exigen la cabecera `X-API-Key`.
- **Reinicios:** `restart: unless-stopped` hace que los servicios vuelvan solos si se reinicia la VM.

- **Volver atrás:** basta cambiar `IMAGE_TAG` por una etiqueta anterior.

12. Herramientas para probar el sistema

El sistema es un servicio sin pantalla propia. Para probarlo hay dos caminos que usan la misma API pública; la guía completa está en el `README.md` del repositorio.

Aplicación de escritorio (`make gui-run`)

- **Partidas:** lista en vivo y detalle con estadísticas, patrones y avisos.
- **Generar datos:** fixtures, escenario demo o carga, con progreso.
- **Verificación:** 37 comprobaciones e informe en Markdown.
- **Enviar evento:** plantillas válidas e inválidas del contrato.
- **Rechazos** y una **Guía** para el evaluador.

Terminal

- `simulator --fixture all --expect`: compara las 8 partidas con lo calculado a mano.
- `make demo`: escenario con casos especiales.
- `curl …/results | jq`: consultar cualquier resultado.
- `scripts/pruebas.sh`: armar partidas evento por evento.
- `scripts/demo.sh` y `make e2e`.

La aplicación de escritorio se mantiene fuera de la arquitectura de análisis:

- reutiliza los paquetes del simulador;
- corre en la máquina del evaluador;
- no contiene lógica de ningún juego.

13. Limitaciones conocidas

Limitación	Por qué se aceptó	Cómo se mejoraría
Un solo nodo de ingestión	Suficiente para la escala del curso; mantiene el confinamiento simple	Particionar partidas por hash entre instancias
Partidas en curso y lotes fallidos solo en memoria	Evita una base de datos; los resultados finales sí persisten	Un registro de eventos en disco o una cola persistente
Los eventos muy tardíos no reabren el análisis	Comportamiento simple y coherente (ADR-06); quedan visibles	Reanálisis incremental con versión del resultado
Autenticación por una sola clave de API	El enunciado excluye la gestión de usuarios	Tokens por productor
Despliegue manual	El CD de imágenes es automático; el despliegue es un clic	Despliegue automático tras `release.yml`

14. Preguntas probables en la defensa

Pregunta	Respuesta corta
¿Go calcula alguna estadística?	No. Solo cuenta eventos aceptados y rechazados para diagnóstico. Guarda el resultado de Scala como bytes, sin leerlo.
¿Dónde está la concurrencia real?	Un dispatcher, una goroutine por partida y un pool de análisis. Una prueba demuestra que un análisis lento de una partida no frena la ingestión de otra, y todo se prueba con `-race`.
¿Por qué no hay mutex en el mapa de partidas?	Porque una sola goroutine lo toca: confinamiento. El único mutex está en el store de resultados.
¿Qué pasa si los eventos llegan desordenados?	Se ordenan por `(timestamp, seq, llegada)`. F06 es F01 desordenado y da exactamente el mismo resultado.
¿Y si `MATCH_FINISHED` llega antes que otros eventos?	Hay 2 s de gracia para los rezagados. Después, `409 LATE_EVENT`, visible en `/v1/rejections`.
¿Y si llega dos veces el mismo evento?	Contenido idéntico: `200 DUPLICATE` (no se cuenta dos veces). Contenido distinto: `409 DUPLICATE_ID_CONFLICT`.
¿Y si Scala se cae?	Tres reintentos con backoff; luego `FAILED` con el lote guardado, y `POST …/reanalyze` lo recupera. Lo prueba el e2e.
¿Qué se pierde al reiniciar Go?	Las partidas en curso y los lotes fallidos. Los resultados analizados persisten en el volumen, y una partida restaurada no se reabre.
¿Por qué la racha no es un simple conteo?	Depende del orden: F12 tiene las mismas 3 eliminaciones que F01, pero una muerte en medio rompe el patrón. Se calcula con un `foldLeft` cuyo estado son las rachas abiertas.
¿Cómo prueban que el análisis es puro?	50 permutaciones de llegada por partida producen exactamente el mismo resultado.
¿Por qué `null` y no 0?	Un 0 sería un dato falso: la precisión de quien nunca atacó no es 0, simplemente no existe.
¿Cómo se agrega un juego?	Un esquema, un `GameAnalyzer`, una línea en el registro y un fixture. El commit de `street-racing` lo demuestra.
¿Por qué no Kafka?	REST cumple el requisito con menos infraestructura. La idempotencia y los reintentos cubren las fallas que tendría que resolver la cola.